

QCM (15 points)

IMPORTANT :

- Une ou plusieurs réponses par question
 - Si plus d'une réponse le nombre est indiqué dans la question
 - Pour les questions à réponses multiples il faut indiquer toutes les réponses pour réussir la question
- Bonne réponse = 0,5 points
- Pas de réponse = 0 point
- Mauvaise réponse = - 0,25 point (vous perdez des points)
- Vous devez répondre sur le feuillet réponse et non sur le questionnaire

La note minimum possible est 0.

Énoncé A : Le jeu multi-joueurs

Les questions 1 à 7 font référence à cet énoncé

Vous devez concevoir un jeu multi-joueurs. Chaque joueur possède un personnage. Pour des raisons de simplicité nous ne considérerons pas la gestion de tout ce qui est réseau/internet et problèmes associés.

Question 1 (voir énoncé A)

Le jeu se déroule dans une maison. La maison est composée de pièces. Quel pattern peut être utilisé pour représenter la maison ?

- | | |
|----------------|--------------|
| a. Composite | f. Singleton |
| b. Décorateur | g. Stratégie |
| c. Etat | h. Visiteur |
| d. Builder | i. MVC |
| e. Observateur | j. Objet |
-

Question 2 (voir énoncé A)

La maison (l'objet qui la représente) doit être informé lorsqu'un personnage interagit avec l'un de ses composants. Quel pattern répond à ce besoin ?

- | | |
|----------------|--------------|
| a. Composite | f. Singleton |
| b. Décorateur | g. Stratégie |
| c. Etat | h. Visiteur |
| d. Builder | i. MVC |
| e. Observateur | j. Objet |

Question 3 (voir énoncé A)

Lorsque le joueur crée son personnage il doit construire son apparence (cheveux, vêtement ...). Quel pattern répond à ce besoin ?

- | | |
|----------------|--------------|
| a. Composite | f. Singleton |
| b. Décorateur | g. Stratégie |
| c. Etat | h. Visiteur |
| d. Builder | i. MVC |
| e. Observateur | j. Objet |

Question 4 (voir énoncé A)

Lorsqu'un joueur débute une partie il doit sélectionner la mission qu'il souhaite accomplir. En fonction de la mission les objectifs à attendre ne sont pas les mêmes. Quel pattern permettra au logiciel que vous développez de gérer ces missions ?

- | | |
|----------------|--------------|
| a. Composite | f. Singleton |
| b. Décorateur | g. Stratégie |
| c. Etat | h. Visiteur |
| d. Builder | i. MVC |
| e. Observateur | j. Objet |

Question 5 (voir énoncé A)

Le jeu comprend une interface graphique. Quel pattern permet de gérer cela ?

- | | |
|----------------|--------------|
| a. Composite | f. Singleton |
| b. Décorateur | g. Stratégie |
| c. Etat | h. Visiteur |
| d. Builder | i. MVC |
| e. Observateur | j. Objet |

Question 6 (voir énoncé A)

Revenons à notre maison. Comme indiqué la maison est composée de pièces. Dans le code, lorsque l'on va instancier des objets pièces, on veut pouvoir faire des appels à la chaîne pour ajouter le mobilier dans la pièce. Par exemple : Chambre () avec 1 lit () avec une commode () avec un bureau() avec une chaise()

Quel pattern permet de gérer cela ?

- | | |
|----------------|--------------|
| a. Composite | f. Singleton |
| b. Décorateur | g. Stratégie |
| c. Etat | h. Visiteur |
| d. Builder | i. MVC |
| e. Observateur | j. Objet |

Question 7 (voir énoncé A)

Un joueur est soit : actif, absent ou déconnecté

Quel pattern permet de gérer cela ?

- | | |
|----------------|--------------|
| a. Composite | f. Singleton |
| b. Décorateur | g. Stratégie |
| c. Etat | h. Visiteur |
| d. Builder | i. MVC |
| e. Observateur | j. Objet |

Question 8

Une méthode particulière est appelée lorsque le « Garbage Collector » s'occupe de supprimer une instance. Quelle affirmation est vraie ?

- a. La signature de la méthode est « public void finalise(); » et on ne sait pas quand elle sera appelée
- b. La signature de la méthode est « public void finalise(); » et on sait quand elle sera appelée
- c. La signature de la méthode est « public void final(); » et on ne sait pas quand elle sera appelée
- d. La signature de la méthode est « public void final(); » et on sait quand elle sera appelée

Question 9

Quelle classe n'a pas de classe mère ?

- a. Orpheline
- b. String
- c. Object
- d. Une classe abstraite

Question 10

Lequel des éléments suivants n'est pas un concept POO en Java?

- a. Héritage
- b. Encapsulation
- c. Polymorphisme
- d. Compilation

Question 11

Toute instance de Y peut appeler la méthode aMethod () de toute autre instance de Y :

```
package a;  
class X{  
public void aMethod(){ }  
}
```

```
package b;  
class Y extends X{ }
```

- a. Vrai
- b. Faux

Question 12

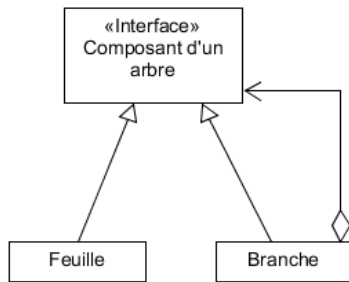
Qu'est ce qui sera stocké dans l'objet « obj » dans la ligne de code suivante?

```
MaClass obj;
```

- a. La valeur null
- b. Pointeur
- c. Adresse mémoire de la mémoire allouée pour l'objet
- d. Un Object

Question 13

Soit le schéma suivant :



Une branche peut contenir :

- a. Une feuille
- b. Plusieurs feuilles
- c. Une feuille et une branche
- d. Plusieurs feuilles et plusieurs branches

Question 14

Quel pattern devons-nous utiliser pour modéliser un jeu de carte ?

- a. Composite
- b. Décorateur
- c. Etat
- d. Builder
- e. Observateur
- f. Singleton
- g. Stratégie
- h. Visiteur
- i. MVC
- j. Objet

Question 15

Quelle est la bonne manière de lancer une exception de type `IllegalArgumentException` signalant un nombre négatif ?

a.	<pre>Exception e = new IllegalArgumentException("nombre negatif"); throw e;</pre>
b.	<pre>try { ... } catch (new IllegalArgumentException("nombre negatif")) { System.out.println(e.getMessage()); }</pre>
c.	<pre>try { ... } catch (IllegalArgumentException e) { e.setMessage("nombre negatif"); }</pre>

Question 16

Observez le morceau de code suivant :

```
public class Company{  
    public abstract double calculateSalaries();  
}
```

Quelle affirmation est dès lors correcte ?

- a. Les mots clés abstract et public ne peuvent pas être utilisés ensemble.
- b. La méthode calculateSalaries dans la classe Company doit avoir un body.
- c. Une instruction return doit se trouver dans la méthode calculateSalaries.
- d. La classe Company doit être définie comme abstraite.

Question 17

Un attribut de classe :

- a. Est forcément abstrait
- b. Est commun à toutes les instances
- c. Est dupliqué dans chaque instance
- d. N'est pas accessible depuis une instance de la classe

Question 18

Quelle est la manière correcte de déclarer que la méthode readName peut causer une IOException ?

a.	<pre>import java.io.*; ... public String readName(Reader r) { throws new IOException(); ... }</pre>
b.	<pre>import java.io.*; ... public String readName(Reader r) throws IOException { ... }</pre>
c.	<pre>import java.io.*; ... public IOException readName(Reader r) { ... }</pre>

Question 19

Vous devez écrire un programme qui gère des demandes clients. Chaque demande à un type et selon ce type un traitement approprié doit être exécuté. Quel pattern sera le mieux adapté pour répondre à cette problématique ?

- | | |
|----------------|--------------|
| a. Composite | f. Singleton |
| b. Décorateur | g. Stratégie |
| c. Etat | h. Visiteur |
| d. Builder | i. MVC |
| e. Observateur | j. Objet |

Question 20

Lequel est un exemple de polymorphisme ?

- A. Les classes internes
- B. Classes anonymes
- C. La surcharge de méthode
- D. La redéfinition de méthode

Question 21

La relation entre une voiture et son propriétaire ou un compte de banque et le client est :

- a. Relation d'agrégation
- b. Relation d'héritage
- c. Relation de composition
- d. Relation d'association

Question 22

Quel est la bonne façon de déclarer un objet monObj de la classe MaClasse ?

- a. `new monObj = MaClasse();`
- b. `class MaClasse = new monObj();`
- c. `MaClasse monObj = new MaClasse();`
- d. `class monObj = new MaClasse();`

Question 23

Dans le pattern MVC, quelle est la couche métier ?

- a. Modèle
- b. Vue
- c. Contrôleur
- d. Client

Question 24

Soit le code suivant :

```
public class Test{

    public static void main(String[] args) {
        // Votre réponse
    }

interface Position{}
class Premier{}
class Second extends Premier{}
class Troisieme extends Premier{}
class Quatrieme extends Second implements Position{}
class Cinquieme extends Quatrieme{}
class Sixieme extends Troisieme{}
```

Quelles instructions pouvez mettre pour remplacer la ligne « // Votre réponse » qui ne causera pas d'erreur de compilation ou d'exécution ?

Note : il y a 2 réponses à cette question

- a. Position pos = new Sixieme();
- b. Premier premier = new Cinquieme();
- c. Position pos = new Quatrieme();
- d. Troisieme troisieme = new Quatrieme();

Question 25

Pour la classe définie comme suit :

```
public class Test {int x ;} ;
```

Lequel des programmes est faux ?

- a. Test a=new Test(3) ;
- b. Test a=new Test () ; String s=a.toString() ;
- c. Test a=new Test () ; boolean b= a.equals("bonjour");
- d. Test a=new Test () ; boolean b= a.equals(a) ;

Question 26

De quelle relation s'agit-il ?



- a. composition
- b. héritage
- c. agrégation
- d. association

Question 27

Parmi les situations suivantes quels sont celles où l'utilisation du pattern Builder est préconisée ?

Note : il y a 3 réponses à cette question

- a. L'objet final est imposant, et sa création complexe
- b. Beaucoup d'arguments doivent être passés à la construction de l'objet
- c. Les arguments doivent être correctement instanciés
- d. Certains des arguments sont optionnels

Question 28

Parmi les arguments suivants lesquels sont en faveur du typage fort ?

Note : il y a 3 réponses à cette question

- a. Avec la vérification de type, la plupart des erreurs sont détectées à la compilation
- b. Cette approche permet de créer des composants logiciels hautement réutilisables au détriment de la détection de certaines erreurs à l'exécution

- c. La déclaration de variables typées peut servir à produire de la documentation
- d. Connaissant le type d'une variable, le compilateur peut effectuer des optimisations

Question 29

Pour lesquelles de ces problématiques le pattern observateur est-il le mieux adapté?

Note : il y a 2 réponses à cette question

- a. Modifier l'apparence d'un objet existant
- b. Dans un tableur, lorsque la valeur d'une case change, recalculer la somme des valeurs
- c. Selon la valeur en paramètre (1, 2, ...) lancer l'algorithme correspondant
- d. Lorsque l'utilisateur click avec la souris sur le bouton de notre application alors lancer l'algorithme

Question 30

L'opérateur this est :

- a. Une référence sur une variable publique
- b. Une référence sur la classe
- c. Une référence sur une variable locale
- d. Une référence sur l'objet en cours

Conception (5 points)

Pour cet exercice vous devez produire un schéma pour répondre à la problématique ci-dessous.

- Votre solution doit mettre en œuvre les notions orientés objets vu dans le cours (interface, classe abstraite, héritage, ...) (1 point)
- Votre solution doit mettre en œuvre un minimum de 3 design patterns, soit 1 par point d'attention plus bas. (1 point par pattern)
- Il faut identifier chaque pattern sur le schéma (si vous ne le faites pas il n'y a pas de point attribué)
- Le pattern Singleton est hors scope. Vous pouvez l'utiliser mais il ne compte pas comme un des 3 patterns
- 1 point est accordé pour la cohérence et la logique globale de votre solution :
 - o Solution logique et cohérente : 1 point
 - o Solution logique et cohérente malgré quelques petites erreurs : 0,5 point
 - o Solution comportant une ou plusieurs erreurs importantes : 0 point

Problématique

Vous devez réaliser un logiciel pour simuler le fonctionnement d'un aquarium. Dans cet aquarium nous trouvons :

- Des plantes

- Des poissons
- Des crustacés (c'est un aquarium luxueux !!)
- Des roches
- Des bouts de bois
- Des abris pour poissons

Le logiciel affichera une image sur lequel nous verrons l'aquarium, les poissons, les plantes ainsi que les autres éléments indiqués.

Il vous faut faire une modélisation objet complète de l'aquarium et des éléments listés précédemment.

Vous devez également répondre aux points d'attention suivants :

Point d'attention #1 : Chaque objet doit être informé lorsqu'un autre objet le touche. Cela pour permettre au logiciel d'adapter le mouvement des objets touchés sur l'image. Par exemple un poisson pousse une algue. Utilisez un pattern vu dans le cours pour répondre à ce point d'attention.

Point d'attention #2 : L'image devra s'adapter en fonction de l'heure de la journée. Les éléments seront bien éclairés le jour et sombre la nuit. Utilisez un pattern vu dans le cours pour répondre à ce point d'attention.

Point d'attention #3 : Il faut pouvoir « créer » des combinaisons d'éléments dans l'aquarium. Des exemples de combinaisons sont :

- Une roche, de couleur grise, avec des algues sur le dessus
- Une roche, de couleur noire, avec un abri sur le dessus, et des algues sur l'abri

Utilisez un pattern vu dans le cours pour répondre à ce point d'attention.